

VL-10: LOOP und WHILE Programme II

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Wiederholung

Wdh.: Programmiersprachen LOOP und WHILE

- Variablen: x_0 x_1 x_2 x_3 ...
- Konstanten: 0 und 1
- Symbole: $:=$ + ;
- Schlüsselwörter: LOOP DO ENDLOOP

- Variablen: x_0 x_1 x_2 x_3 ...
- Konstanten: 0 und 1
- Symbole: $:=$ + ; $\neq$
- Schlüsselwörter: LOOP DO ENDLOOP **WHILE ENDWHILE**

Ein LOOP Programm P mit k Variablen
berechnet eine k -stellige Funktion der Form $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.

Ist P die Zuweisung $x_i := x_j + c$,
so ist $[P](r_0, \dots, r_{k-1}) = (r_0, \dots, r_{i-1}, r_j + c, r_{i+1}, \dots, r_{k-1})$.

Ist $P = P_1; P_2$ eine Hintereinanderausführung,
so ist $[P](r_0, \dots, r_{k-1}) = [P_2]([P_1](r_0, \dots, r_{k-1}))$.

Ist $P = \text{LOOP } x_i \text{ DO } Q \text{ ENDLOOP}$ ein LOOP-Konstrukt,
so gilt $[P](r_0, \dots, r_{k-1}) = [Q]^{r_i}(r_0, \dots, r_{k-1})$.

Ein WHILE Programm P mit k Variablen
berechnet eine k -stellige Funktion der Form $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.

Ist $P = \text{WHILE } x_i \neq 0 \text{ DO } Q \text{ ENDWHILE}$ ein WHILE-Konstrukt,
so ist $[P](r_0, \dots, r_{k-1}) = [Q]^\ell(r_0, \dots, r_{k-1})$ für die kleinste Zahl ℓ ,
für die die i -te Komponente von $[Q]^\ell(r_0, \dots, r_{k-1})$ gleich 0 ist.

Falls solch ein ℓ nicht existiert, so ist $[P](r_0, \dots, r_{k-1})$ undefiniert.

Wdh.: WHILE versus LOOP

- Jede LOOP-berechenbare Funktion $f: \mathbb{N}^k \rightarrow \mathbb{N}$ ist auch WHILE-berechenbar.
- Es gibt WHILE Programme, die nicht terminieren.
LOOP Programme terminieren immer.

Satz

Die Programmiersprache WHILE ist Turing-mächtig.

Wdh.: WHILE ist Turing-mächtig

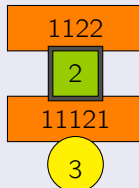
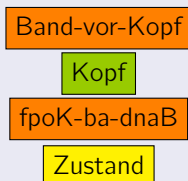
Simulierte Turingmaschine M :



δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Die vier entsprechenden Variablen im WHILE Programm:



Variable **BandLinks**
Variable **UntermKopf**
Variable **BandRechts**
Variable **Zustand**

Vorlesung VL-10

LOOP und WHILE Programme II

- Die Ackermann Funktion
- Eigenschaften der Ackermann Funktion
- Ackermann Funktion und LOOP Programme
- Primitive Rekursion

Vermutung von David Hilbert (1926)

Die Menge der LOOP-berechenbaren Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}$ stimmt mit der Menge der berechenbaren totalen Funktionen überein.

(Als Teil von Hilberts Programm, mit “finiten” Methoden die Widerspruchsfreiheit der Axiomensysteme der Mathematik nachzuweisen.)

Vermutung von David Hilbert (1926)

Die Menge der LOOP-berechenbaren Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}$ stimmt mit der Menge der berechenbaren totalen Funktionen überein.

(Als Teil von Hilberts Programm, mit “finiten” Methoden die Widerspruchsfreiheit der Axiomensysteme der Mathematik nachzuweisen.)

Satz (Ackermann, 1926/1928)

Es existieren berechenbare totale Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}$, die nicht LOOP-berechenbar sind.

Wilhelm Ackermann (1896–1962)

Wikipedia: **Wilhelm Friedrich Ackermann** war ein deutscher Mathematiker. Schüler von David Hilbert. Promotion 1924 in Göttingen.

1926 entdeckte er die nach ihm benannte Ackermann Funktion, die in der Berechenbarkeitstheorie eine wichtige Rolle spielt. Ackermann war Gymnasiallehrer an verschiedenen Schulen in Münster, Burgsteinfurt und Lüdenscheid; er hielt auch Vorlesungen an der Universität Münster.

D. Hilbert und W. Ackermann:
“Grundzüge der theoretischen Logik”
Springer Verlag, 1928



(Hilbert über Ackermann: “Oh, das ist wunderbar. Das sind gute Neuigkeiten für mich. Denn wenn dieser Mann so verrückt ist, dass er heiratet und sogar ein Kind hat, bin ich von jeder Verpflichtung befreit, etwas für ihn tun zu müssen.”)

Die Ackermann Funktion

Ackermann Funktion: Definition

Definition

Die Ackermann Funktion $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ ist folgendermassen definiert:

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Wilhelm Ackermann:

“Zum Hilbertschen Aufbau der reellen Zahlen”

Mathematische Annalen 99 (1928), pp. 118–133

Rózsa Péter:

“Konstruktion nichtrekursiver Funktionen”

Mathematische Annalen 111 (1935), pp. 42–60

Ackermann Funktion: Einige Beispiele

Wenn man den ersten Parameter fixiert:

- $A(1, n) = n + 2$
- $A(2, n) = 2n + 3$
- $A(3, n) = 2^{n+3} - 3$
- $A(4, n) = \underbrace{2^{2^{\cdot^{\cdot^2}}}}_{\substack{n+3 \text{ viele} \\ \text{Zweien}}} - 3$

Bereits der Wert $A(4, 2) = 2^{65536} - 3$
ist grösser als die Anzahl aller Atome im Weltraum.

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

- Multiplikation ist iterierte Addition:

$$\underbrace{a + \dots + a}_{b \text{ mal}} = a \cdot b$$

- Potenzierung ist iterierte Multiplikation:

$$\underbrace{a \times \dots \times a}_{b \text{ mal}} = a^b =: a \uparrow b$$

- Der Potenzturm ist iterierte Potenzierung:

$$\underbrace{a^{a^{\dots^a}}}_{b \text{ mal}} =: a \uparrow\uparrow b$$

- Wiederholte Potenzturmbildung

gibt $\underbrace{a \uparrow\uparrow a \uparrow\uparrow \dots \uparrow\uparrow a}_{b \text{ mal}} =: a \uparrow\uparrow\uparrow b$

Exkurs: UpArrow-Notation (2)

Definition (UpArrow-Notation)

$$a \uparrow^m b := \begin{cases} 1 & \text{wenn } b = 0 \\ a \cdot b & \text{wenn } m = 0 \\ a^b & \text{wenn } m = 1 \\ a \uparrow^{m-1} (a \uparrow^m (b-1)) & \text{sonst} \end{cases}$$

Es gilt:

- $A(1, n) = 2 + (n + 3) - 3$
- $A(2, n) = 2 \cdot (n + 3) - 3$
- $A(3, n) = 2 \uparrow (n + 3) - 3$
- $A(4, n) = 2 \uparrow\uparrow (n + 3) - 3$
- $A(5, n) = 2 \uparrow\uparrow\uparrow (n + 3) - 3$
- $\vdots$
- $A(m, n) = 2 \uparrow^{m-2} (n + 3) - 3 \quad \text{für } m \geq 2$

Berechenbarkeit der Ackermann Funktion

Die Ackermann Funktion ist berechenbar

$$\begin{array}{lll} A(0, n) & = & n + 1 \quad \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) \quad \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) \quad \text{für } m, n \geq 0 \end{array}$$

Satz

Die Ackermann Funktion ist Turing-berechenbar.

Die Ackermann Funktion ist berechenbar

$$\begin{array}{lll} A(0, n) & = & n + 1 \quad \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) \quad \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) \quad \text{für } m, n \geq 0 \end{array}$$

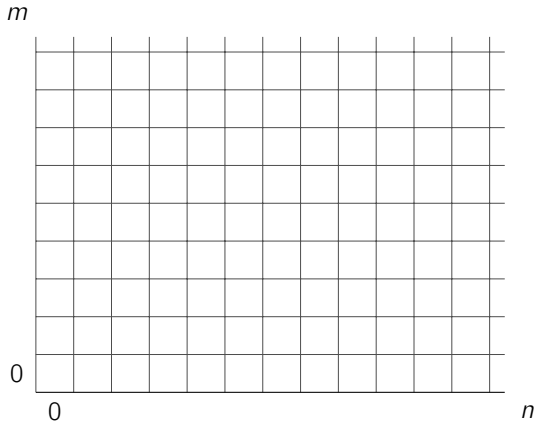
Satz

Die Ackermann Funktion ist Turing-berechenbar.

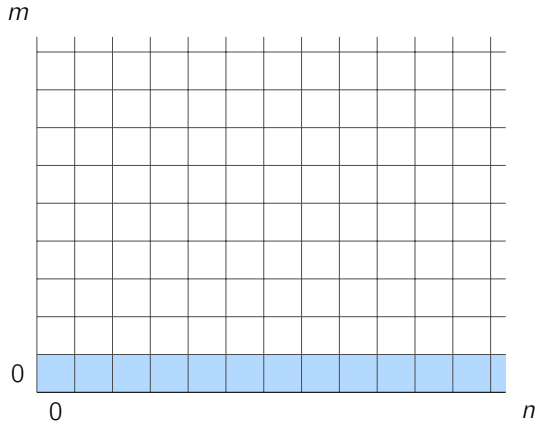
Beweis:

- Wir zeigen mit Induktion über $m \geq 0$, dass jede Funktion $f_m : \mathbb{N} \rightarrow \mathbb{N}$ mit $f_m(x) = A(m, x)$ berechenbar ist.
- Für $m = 0$ ist f_0 die Nachfolgerfunktion $f_0(x) = x + 1$.
- Für $m \geq 1$ berechnen wir $f_m(x)$, indem wir der Reihe nach induktiv alle Werte $f_m(0)$, $f_m(1)$, $f_m(2)$, $\dots$, $f_m(x - 1)$ bestimmen.
- Zum Schluss berechnen wir:
$$f_m(x) = A(m, x) = A(m - 1, A(m, x - 1)) = f_{m-1}(f_m(x - 1))$$

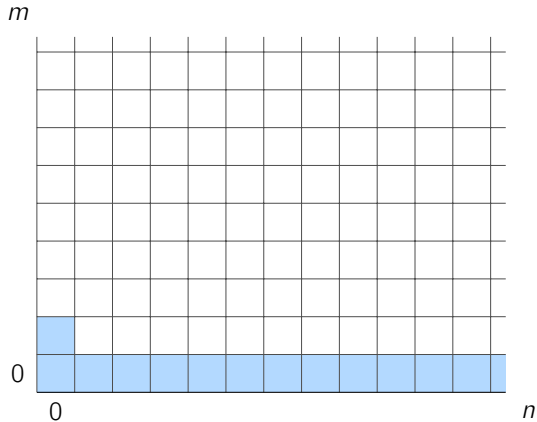
Induktion über lexikographisch sortierte Paare (m, n)



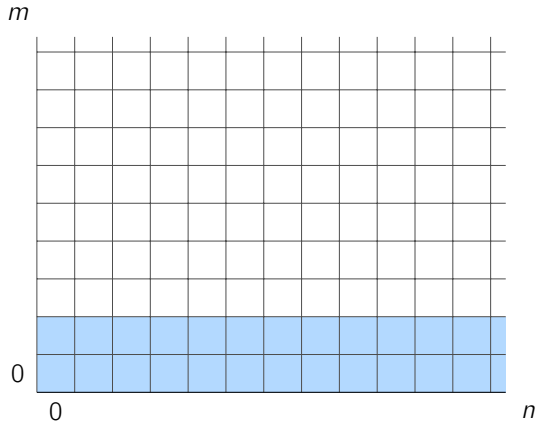
Induktion über lexikographisch sortierte Paare (m, n)



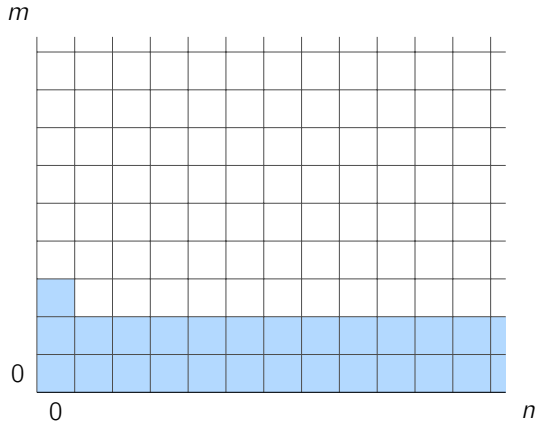
Induktion über lexikographisch sortierte Paare (m, n)



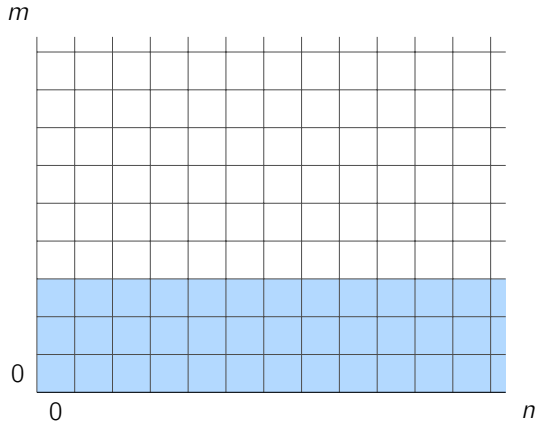
Induktion über lexikographisch sortierte Paare (m, n)



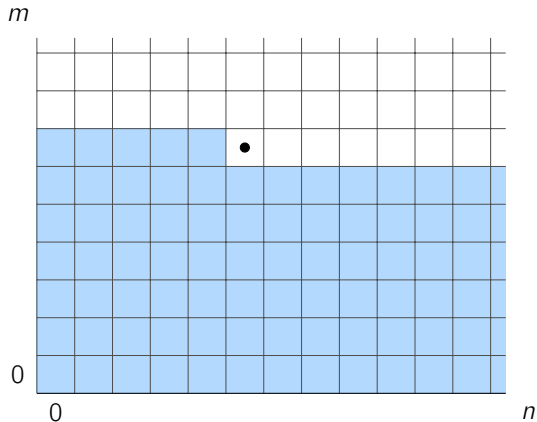
Induktion über lexikographisch sortierte Paare (m, n)



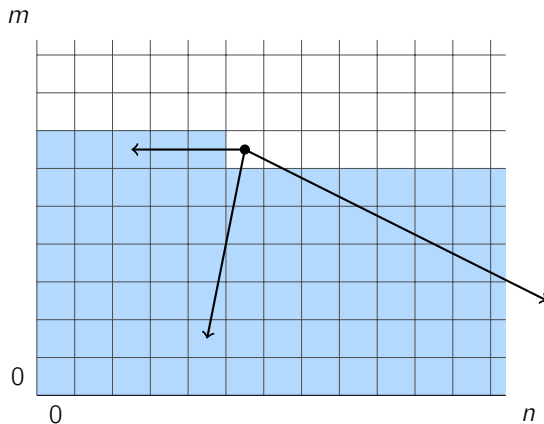
Induktion über lexikographisch sortierte Paare (m, n)



Induktion über lexikographisch sortierte Paare (m, n)



Induktion über lexikographisch sortierte Paare (m, n)



Monotonie-Verhalten der Ackermann Funktion

Monotonie

$$(M.1) \quad A(m, n + 1) > A(m, n)$$

$$(M.2) \quad A(m + 1, n) > A(m, n)$$

$$(M.3) \quad A(m + 1, n - 1) \geq A(m, n)$$

Monotonie

$$(M.1) \quad A(m, n+1) > A(m, n)$$

$$(M.2) \quad A(m+1, n) > A(m, n)$$

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Einfache Folgerung aus (M.1) und (M.2)

Für $m \geq m'$ und $n \geq n'$ gilt immer:

$$A(m, n) \geq A(m', n')$$

Wenn $(m, n) \neq (m', n')$ gilt sogar strikte Ungleichheit.

Monotonie-Beweis (M.1)

$$\begin{array}{lll} A(0, n) & = & n + 1 \quad \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) \quad \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) \quad \text{für } m, n \geq 0 \end{array}$$

Wir wollen nun Ungleichung (M.1) zeigen: $A(m, n + 1) > A(m, n)$

Monotonie-Beweis (M.1)

$$\begin{array}{lll} A(0, n) & = & n + 1 \quad \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) \quad \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) \quad \text{für } m, n \geq 0 \end{array}$$

Wir wollen nun Ungleichung (M.1) zeigen: $A(m, n + 1) > A(m, n)$

Die Ackermann'sche Rekursionsgleichung liefert:

$$\begin{aligned} A(m, n + 1) &= A(m - 1, A(m, n)) \\ A(m, n) &= A(m - 1, A(m, n - 1)) \end{aligned}$$

Monotonie-Beweis (M.1)

$$\begin{array}{lll} A(0, n) & = & n + 1 \quad \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) \quad \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) \quad \text{für } m, n \geq 0 \end{array}$$

Wir wollen nun Ungleichung (M.1) zeigen: $A(m, n + 1) > A(m, n)$

Die Ackermann'sche Rekursionsgleichung liefert:

$$\begin{aligned} A(m, n + 1) &= A(m - 1, A(m, n)) \\ A(m, n) &= A(m - 1, A(m, n - 1)) \end{aligned}$$

Induktion liefert zuerst $x := A(m, n) > A(m, n - 1) =: y$
und danach $A(m - 1, x) > A(m - 1, y)$.

Ein analoges induktives Argument beweist die Ungleichung (M.2).

Übung

Beweisen Sie die Ungleichung (M.2) mit vollständiger Induktion über die lexikographisch sortierten Paare (m, n) .

$$A(m+1, n) > A(m, n) \quad \text{für alle } m, n \geq 0.$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$A(m+1, n-1)$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$A(m+1, n-1) = A(m, A(m+1, n-2)) \quad (\text{Def})$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$\begin{aligned} A(m+1, n-1) &= A(m, A(m+1, n-2)) && \text{(Def)} \\ &\geq A(m, A(m, n-1)) && \text{(Ind+Mono)} \end{aligned}$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$\begin{aligned} A(m+1, n-1) &= A(m, A(m+1, n-2)) && \text{(Def)} \\ &\geq A(m, A(m, n-1)) && \text{(Ind+Mono)} \\ &\geq A(m-1, A(m, n-1)+1) && \text{(Ind)} \end{aligned}$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$\begin{aligned} A(m+1, n-1) &= A(m, A(m+1, n-2)) && \text{(Def)} \\ &\geq A(m, A(m, n-1)) && \text{(Ind+Mono)} \\ &\geq A(m-1, A(m, n-1) + 1) && \text{(Ind)} \\ &\geq A(m-1, A(m, n-1)) && \text{(Mono)} \end{aligned}$$

Monotonie-Beweis (M.3)

$$(M.3) \quad A(m+1, n-1) \geq A(m, n)$$

Induktionsanfang: Es gilt $A(1, n-1) = n+1 = A(0, n)$ für alle $n \geq 1$.
Es gilt $A(m+1, 0) = A(m, 1)$ für alle $m \geq 0$.

Induktionsannahme: Wir nehmen an, dass für (m', n') mit $(m' < m)$ oder $(m' = m) \wedge (n' < n)$ immer $A(m'+1, n'-1) \geq A(m', n')$ gilt.

$$\begin{aligned} A(m+1, n-1) &= A(m, A(m+1, n-2)) && \text{(Def)} \\ &\geq A(m, A(m, n-1)) && \text{(Ind+Mono)} \\ &\geq A(m-1, A(m, n-1) + 1) && \text{(Ind)} \\ &\geq A(m-1, A(m, n-1)) && \text{(Mono)} \\ &= A(m, n) && \text{(Def)} \end{aligned}$$

Das Wachstumslemma

Wachstumslemma: Vorbereitungen (1)

Wir werden in diesem Kapitel ausschliesslich LOOP Programme betrachten, die die folgende technische Annahme erfüllen:

Technische Annahme:

In LOOP-Konstrukten `LOOP  $x_i$  DO  $P$  ENDLOOP` kommt die Zählvariable x_i nicht im inneren Programmteil P vor.

Andernfalls führen wir für die Schleife eine frische Zählvariable x'_i ein:

```
 $x'_i := x_i;$   
LOOP  $x'_i$  DO  $P$  ENDLOOP
```

Wachstumslemma: Vorbereitungen (2)

- Wir betrachten ein fixes LOOP Programm P mit k Variablen.
- Wir erinnern uns (aus VL-09):
Das Programm P berechnet die k -stellige Funktion $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.
Das LOOP Programm P übersetzt also einen Eingabevektor $\vec{a} = (a_1, \dots, a_k) \in \mathbb{N}^k$ in einen Ausgabevektor $(b_1, \dots, b_k)$.

Wachstumslemma: Vorbereitungen (2)

- Wir betrachten ein fixes LOOP Programm P mit k Variablen.
- Wir erinnern uns (aus VL-09):
Das Programm P berechnet die k -stellige Funktion $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.
Das LOOP Programm P übersetzt also einen Eingabevektor $\vec{a} = (a_1, \dots, a_k) \in \mathbb{N}^k$ in einen Ausgabevektor $(b_1, \dots, b_k)$.

Definition

Für die Anfangswerte $\vec{a} = (a_1, \dots, a_k) \in \mathbb{N}^k$ der Variablen definieren wir

$$f_P(\vec{a}) := b_1 + b_2 + \dots + b_k$$

als die Summe aller Ergebniswerte in $[P](\vec{a}) = (b_1, \dots, b_k)$.

Wachstumslemma: Vorbereitungen (3)

Definition

Die Wachstumsfunktion $F_P: \mathbb{N} \rightarrow \mathbb{N}$ ist definiert durch

$$F_P(n) = \max \left\{ f_P(\vec{a}) \mid \vec{a} \in \mathbb{N}^k \text{ mit } \sum_{i=1}^k a_i \leq n \right\}$$

Intuition:

- Die Funktion F_P beschreibt den maximalen Wert, auf den die Variablensumme durch das LOOP Programm P anwachsen kann.
- Wenn wir das Programm mit einer Variablensumme $\leq n$ füttern, so liefert uns das Programm eine Variablensumme von höchstens $F_P(n)$.

Wachstumslemma: Zentrale Aussage

Wir werden nun zeigen,
dass für jedes LOOP Programm P und für alle $n \in \mathbb{N}$
der Wert $F_P(n)$ echt kleiner ist als der Wert $A(m, n)$,
falls der Parameter m nur gross genug gewählt wird.

Wachstumslemma: Zentrale Aussage

Wir werden nun zeigen,
dass für jedes LOOP Programm P und für alle $n \in \mathbb{N}$
der Wert $F_P(n)$ echt kleiner ist als der Wert $A(m, n)$,
falls der Parameter m nur gross genug gewählt wird.

Lemma (Wachstumslemma)

Für jedes LOOP Programm P existiert eine natürliche Zahl m_P ,
sodass für alle $n \in \mathbb{N}$ gilt: $F_P(n) < A(m_P, n)$.

Beweis: Induktion über Aufbau des Programms

Induktionsanfang (Zuweisungen)

- Es sei P von der Form $x_i := x_j + c$ mit $c \in \{0, 1\}$.
- Wir werden zeigen: $F_P(n) < A(2, n)$.

Induktionsschritt (Hintereinanderausführung)

- Es sei P von der Form $P_1; P_2$.
- Induktionsannahme: $\exists q \in \mathbb{N} : F_{P_1}(\ell) < A(q, \ell)$ und $F_{P_2}(\ell) < A(q, \ell)$.
- Wir werden zeigen: $F_P(n) < A(q + 1, n)$.

Induktionsschritt (LOOP-Konstrukt)

- Es sei P von der Form "LOOP x_i DO Q ENDLOOP"
- Induktionsannahme: $\exists q \in \mathbb{N} : F_Q(\ell) < A(q, \ell)$.
- Wir werden zeigen: $F_P(n) < A(q + 1, n)$.

Beweis: Induktionsanfang

Induktionsanfang (Zuweisungen)

- Es sei P von der Form " $x_i := x_j + c$ " mit $c \in \{0, 1\}$
- Dann gilt: $F_P(n) \leq 2n + 1$, und somit $F_P(n) < A(2, n) = 2n + 3$

Beweis: Induktionsanfang

Induktionsanfang (Zuweisungen)

- Es sei P von der Form “ $x_i := x_j + c$ ” mit $c \in \{0, 1\}$
- Dann gilt: $F_P(n) \leq 2n + 1$, und somit $F_P(n) < A(2, n) = 2n + 3$

Argument:

- P verändert nur den Wert der Variablen x_i .

Beweis: Induktionsanfang

Induktionsanfang (Zuweisungen)

- Es sei P von der Form " $x_i := x_j + c$ " mit $c \in \{0, 1\}$
- Dann gilt: $F_P(n) \leq 2n + 1$, und somit $F_P(n) < A(2, n) = 2n + 3$

Argument:

- P verändert nur den Wert der Variablen x_i .
- Am Anfang war $x_i \geq 0$, und am Ende ist $x_i = x_j + c \leq n + 1$.

Beweis: Induktionsanfang

Induktionsanfang (Zuweisungen)

- Es sei P von der Form " $x_i := x_j + c$ " mit $c \in \{0, 1\}$
- Dann gilt: $F_P(n) \leq 2n + 1$, und somit $F_P(n) < A(2, n) = 2n + 3$

Argument:

- P verändert nur den Wert der Variablen x_i .
- Am Anfang war $x_i \geq 0$, und am Ende ist $x_i = x_j + c \leq n + 1$.
- Die Summe aller Variablenwerte erhöht sich um höchstens $n + 1$, und springt damit
von höchstens n (am Anfang)
auf höchstens $2n + 1$ (am Ende).

Beweis: Schritt für Hintereinanderausführung

Induktionsschritt (Hintereinanderausführung)

- Es sei P von der Form " $P_1; P_2$ "
- Induktionsannahme: $\exists q \in \mathbb{N} : F_{P_1}(\ell) < A(q, \ell)$ und $F_{P_2}(\ell) < A(q, \ell)$.
- Dann gilt: $F_P(n) < A(q + 1, n)$

Beweis: Schritt für Hintereinanderausführung

Induktionsschritt (Hintereinanderausführung)

- Es sei P von der Form " $P_1; P_2$ "
- Induktionsannahme: $\exists q \in \mathbb{N} : F_{P_1}(\ell) < A(q, \ell)$ und $F_{P_2}(\ell) < A(q, \ell)$.
- Dann gilt: $F_P(n) < A(q+1, n)$

Argument:

- Wir verwenden Abschätzung (M.3): $A(q, n) \leq A(q+1, n-1)$
- Dann folgt mit den Monotonie Eigenschaften, dass

$$\begin{aligned} F_P(n) &\leq F_{P_2}(F_{P_1}(n)) && \text{(Def)} \\ &< A(q, A(q, n)) && \text{(Ind+Mono)} \\ &\leq A(q, A(q+1, n-1)) && \text{(Mono+M.3)} \\ &= A(q+1, n) && \text{(Def)} \end{aligned}$$

Beweis: Schritt fürs LOOP-Konstrukt (1)

Induktionsschritt (LOOP-Konstrukt)

- Es sei P von der Form “LOOP x_i DO Q ENDLOOP”
- Induktionsannahme liefert: $\exists q \in \mathbb{N} : F_Q(\ell) < A(q, \ell)$
- Dann gilt: $F_P(n) < A(q + 1, n)$.

Beweis: Schritt fürs LOOP-Konstrukt (1)

Induktionsschritt (LOOP-Konstrukt)

- Es sei P von der Form “LOOP x_i DO Q ENDLOOP”
- Induktionsannahme liefert: $\exists q \in \mathbb{N} : F_Q(\ell) < A(q, \ell)$
- Dann gilt: $F_P(n) < A(q + 1, n)$.

Argument:

- Wir betrachten jenen Wert $\alpha = \alpha(n)$ für die Variable x_i , mit dem der grösstmögliche Funktionswert $F_P(n)$ angenommen wird

Beweis: Schritt fürs LOOP-Konstrukt (1)

Induktionsschritt (LOOP-Konstrukt)

- Es sei P von der Form “LOOP x_i DO Q ENDLOOP”
- Induktionsannahme liefert: $\exists q \in \mathbb{N} : F_Q(\ell) < A(q, \ell)$
- Dann gilt: $F_P(n) < A(q + 1, n)$.

Argument:

- Wir betrachten jenen Wert $\alpha = \alpha(n)$ für die Variable x_i , mit dem der grösstmögliche Funktionswert $F_P(n)$ angenommen wird
- Dann gilt $F_P(n) \leq F_Q(F_Q(\dots F_Q(F_Q(n - \alpha)) \dots)) + \alpha$, wobei die Funktion $F_Q(\cdot)$ hier α -fach ineinander eingesetzt ist.

Beweis: Schritt fürs LOOP-Konstrukt (1)

Induktionsschritt (LOOP-Konstrukt)

- Es sei P von der Form “LOOP x_i DO Q ENDLOOP”
- Induktionsannahme liefert: $\exists q \in \mathbb{N} : F_Q(\ell) < A(q, \ell)$
- Dann gilt: $F_P(n) < A(q + 1, n)$.

Argument:

- Wir betrachten jenen Wert $\alpha = \alpha(n)$ für die Variable x_i , mit dem der grösstmögliche Funktionswert $F_P(n)$ angenommen wird
- Dann gilt $F_P(n) \leq F_Q(F_Q(\dots F_Q(F_Q(n - \alpha)) \dots)) + \alpha$, wobei die Funktion $F_Q(\cdot)$ hier α -fach ineinander eingesetzt ist.
- Die Induktionsannahme gibt uns $F_Q(\ell) < A(q, \ell)$
- Dies wenden wir auf die äusserste Funktion F_Q an und erhalten

$$F_P(n) < A(q, F_Q(\dots F_Q(F_Q(n - \alpha)) \dots)) + \alpha$$

Und weiter geht's:

- Wiederholte Anwendung der Induktionsannahme $F_Q(\ell) < A(q, \ell)$ liefert uns die α -zeilige Ungleichungskette

$$\begin{aligned} F_P(n) &< A(q, F_Q(F_Q(\dots F_Q(F_Q(n - \alpha)) \dots))) + \alpha \\ &< A(q, A(q, F_Q(\dots F_Q(F_Q(n - \alpha)) \dots))) + \alpha \\ &\quad \vdots \\ &< A(q, A(q, A(q, \dots, A(q, n - \alpha)) \dots))) + \alpha \end{aligned}$$

Beweis: Schritt fürs LOOP-Konstrukt (2)

Und weiter geht's:

- Wiederholte Anwendung der Induktionsannahme $F_Q(\ell) < A(q, \ell)$ liefert uns die α -zeilige Ungleichungskette

$$\begin{aligned} F_P(n) &< A(q, F_Q(\dots F_Q(F_Q(n - \alpha)) \dots)) + \alpha \\ &< A(q, A(q, F_Q(\dots F_Q(F_Q(n - \alpha)) \dots)) + \alpha \\ &\quad \vdots \\ &< A(q, A(q, A(q, \dots, A(q, n - \alpha)) \dots)) + \alpha \end{aligned}$$

- Daraus folgt $F_P(n) \leq A(q, A(q, A(q, \dots, A(q, n - \alpha)) \dots))$

(Begründung: Zwischen der linken Seite L und der rechten Seite R in dieser Ungleichungskette liegen $\alpha - 1$ weitere ganze Zahlen. Daher folgt $L \leq R - \alpha$.)

Beweis: Schritt fürs LOOP-Konstrukt (3)

Und zum Schluss:

- Wir haben: $F_P(n) \leq A(q, A(q, A(q, \dots, A(q, n - \alpha)) \dots))$
- Im innersten Teil der rechten Seite wenden wir zunächst (M.3) in der Form $A(q, n - \alpha) \leq A(q + 1, n - \alpha - 1)$ an

$$F_P(n) \leq A(q, A(q, A(q, \dots A(q, A(q + 1, n - \alpha - 1))) \dots))$$

Beweis: Schritt fürs LOOP-Konstrukt (3)

Und zum Schluss:

- Wir haben: $F_P(n) \leq A(q, A(q, A(q, \dots, A(q, n - \alpha)) \dots))$
- Im innersten Teil der rechten Seite wenden wir zunächst (M.3) in der Form $A(q, n - \alpha) \leq A(q + 1, n - \alpha - 1)$ an

$$F_P(n) \leq A(q, A(q, A(q, \dots A(q, A(q + 1, n - \alpha - 1))) \dots))$$

- Dann wenden wir $(\alpha - 1)$ -mal die Definition der Ackermann Funktion in der Form $A(q, A(q + 1, x)) = A(q + 1, x + 1)$ an und arbeiten uns dabei Schritt für Schritt von innen nach aussen vor.
- Dies führt schlussendlich zur gewünschten Ungleichung

$$F_P(n) \leq A(q + 1, n - 2) < A(q + 1, n)$$

Die Mächtigkeit von LOOP

Die Mächtigkeit von LOOP (1)

Satz

Die Ackermann Funktion ist nicht LOOP-berechenbar.

Die Mächtigkeit von LOOP (1)

Satz

Die Ackermann Funktion ist nicht LOOP-berechenbar.

Beweis:

- Zwecks Widerspruchs nehmen wir an, dass die Ackermann Funktion LOOP-berechenbar ist. Dann gibt es auch ein LOOP Programm P , das die Hilfsfunktion $B(n) = A(n, n)$ berechnet.

Die Mächtigkeit von LOOP (1)

Satz

Die Ackermann Funktion ist nicht LOOP-berechenbar.

Beweis:

- Zwecks Widerspruchs nehmen wir an, dass die Ackermann Funktion LOOP-berechenbar ist. Dann gibt es auch ein LOOP Programm P , das die Hilfsfunktion $B(n) = A(n, n)$ berechnet.
- Da die Funktion F_P das Wachstum des Programms P beschränkt, gilt per Definition für alle $n \in \mathbb{N}$: $B(n) \leq F_P(n)$
- Weiters liefert uns das Wachstumslemma eine fixe Zahl m_P , sodass für alle $n \in \mathbb{N}$ gilt: $F_P(n) < A(m_P, n)$

Die Mächtigkeit von LOOP (1)

Satz

Die Ackermann Funktion ist nicht LOOP-berechenbar.

Beweis:

- Zwecks Widerspruchs nehmen wir an, dass die Ackermann Funktion LOOP-berechenbar ist. Dann gibt es auch ein LOOP Programm P , das die Hilfsfunktion $B(n) = A(n, n)$ berechnet.
- Da die Funktion F_P das Wachstum des Programms P beschränkt, gilt per Definition für alle $n \in \mathbb{N}$: $B(n) \leq F_P(n)$
- Weiters liefert uns das Wachstumslemma eine fixe Zahl m_P , sodass für alle $n \in \mathbb{N}$ gilt: $F_P(n) < A(m_P, n)$
- Wenn wir in den beiden roten Ungleichungen $n := m_P$ setzen, so erhalten wir $B(m_P) \leq F_P(m_P) < A(m_P, m_P) = B(m_P)$.
- Widerspruch! Q.E.D.!

Die Mächtigkeit von LOOP (2)

Wir erinnern uns: Die Ackermann Funktion ist Turing-berechenbar.

Satz

Die Ackermann Funktion ist WHILE-berechenbar.

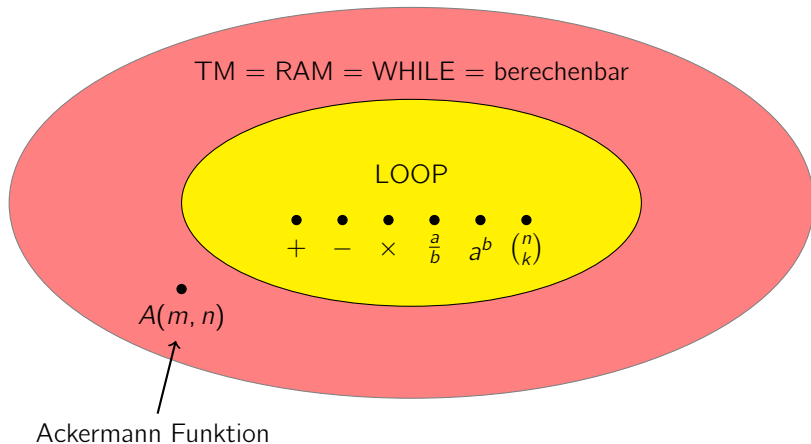
Wir fassen zusammen:

Folgerung

Die Menge der LOOP-berechenbaren Funktionen bildet eine **echte Teilmenge** der berechenbaren Funktionen.

Anders gesagt: Die Programmiersprache LOOP ist nicht Turing-mächtig

Landschaft um LOOP, WHILE, TM, RAM



Primitiv rekursive Funktionen

Primitiv rekursive Funktionen

Die primitiv rekursiven Funktionen bilden eine Unterfamilie der Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}$ mit $k \geq 1$.

Sie sind induktiv definiert, und werden durch zwei **Operationen** aus den sogenannten **Basisfunktionen** zusammengesetzt.

Thoralf Skolem (1923):

*“Begründung der elementaren Arithmetik durch die rekurrierende Denkweise ohne Anwendung scheinbarer Veränderlichen mit unendlichem Ausdehnungsbereich”,
Skrifter utgitt av Videnskapsselskapet i Kristiania 6, pp 1–38.*

Definition (Basisfunktionen)

Die folgenden drei Funktionsmengen bilden die Basisfunktionen unter den primitiv rekursiven Funktionen:

- Alle **konstanten Funktionen** sind primitiv rekursiv.
- Alle identischen Abbildungen (**Projektionen** auf eine der Komponenten) sind primitiv rekursiv.
- Die **Nachfolgerfunktion** $\text{succ}(n) = n + 1$ ist primitiv rekursiv.

Beispiele:

- Konstante Funktion $f : \mathbb{N}^6 \rightarrow \mathbb{N}$ mit $f(a, b, c, d, e, f) \equiv 277$
- Projektion $f : \mathbb{N}^6 \rightarrow \mathbb{N}$ auf vierte Komponente:
 $f(a, b, c, d, e, f) = d$

Anmerkung: Die Projektion auf die p -te von k Komponenten wird manchmal mit $\pi_{k,p}$ bezeichnet.

Operation: Komposition

Definition (Operation)

Jede **Komposition** von primitiv rekursiven Funktionen ist primitiv rekursiv.

Wenn die Funktion $g : \mathbb{N}^k \rightarrow \mathbb{N}$ primitiv rekursiv ist und wenn die k Funktionen $h_1, \dots, h_k : \mathbb{N}^\ell \rightarrow \mathbb{N}$ primitiv rekursiv sind, so ist auch die folgende Funktion $f : \mathbb{N}^\ell \rightarrow \mathbb{N}$ primitiv rekursiv:

$$f(x_1, \dots, x_\ell) = g(h_1(x_1, \dots, x_\ell), \dots, h_k(x_1, \dots, x_\ell))$$

Beispiel:

- Die Funktion $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $f(x, y) = y + 1$ ist primitiv rekursiv, da $f(x, y) = \text{succ}(\pi_{2,2}(x, y))$ gilt.

Operation: Primitive Rekursion

Definition (Operation)

Jede Funktion, die durch **primitive Rekursion** aus primitiv rekursiven Funktionen entsteht ist primitiv rekursiv.

Wenn die beiden Funktionen $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ primitiv rekursiv sind, so ist auch die wie folgt definierte Funktion $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ primitiv rekursiv:

$$\begin{aligned} f(0, x_1, \dots, x_k) &= g(x_1, \dots, x_k) \\ f(n+1, x_1, \dots, x_k) &= h(n, f(n, x_1, \dots, x_k), x_1, \dots, x_k) \end{aligned}$$

Anmerkungen:

- Formal wird die primitive Rekursion immer über die erste Komponente durchgeführt.
- Im Fall $k = 0$ ist f eine Funktion $\mathbb{N} \rightarrow \mathbb{N}$

Intuition zu primitiv rekursiven Funktionen

Beobachtung: Jede primitiv rekursive Funktion ist berechenbar und total.

- Klar für Basisfunktionen. Und die Komposition von berechenbaren und totalen Funktionen ist ebenfalls berechenbar und total.

Intuition zu primitiv rekursiven Funktionen

Beobachtung: Jede primitiv rekursive Funktion ist berechenbar und total.

- Klar für Basisfunktionen. Und die Komposition von berechenbaren und totalen Funktionen ist ebenfalls berechenbar und total.
- Angenommen, die beiden Funktionen $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ sind berechenbar. Dann berechnen wir der Reihe nach:

$$\begin{aligned}y_0 &:= g(x_1, \dots, x_k) &&= f(0, x_1, \dots, x_k) \\y_1 &:= h(0, y_0, x_1, \dots, x_k) &&= f(1, x_1, \dots, x_k) \\y_2 &:= h(1, y_1, x_1, \dots, x_k) &&= f(2, x_1, \dots, x_k) \\y_3 &:= h(2, y_2, x_1, \dots, x_k) &&= f(3, x_1, \dots, x_k) \\&\vdots && \\y_n &:= h(n-1, y_{n-1}, x_1, \dots, x_k) &&= f(n, x_1, \dots, x_k)\end{aligned}$$

Damit haben wir dann auch $f(n, x_1, \dots, x_k) = y_n$ berechnet.

Primitiv rekursive Beispiele

Beispiel: Addition

Die Additionsfunktion $\text{add} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{add}(x, y) = x + y$ ist primitiv rekursiv.

$$\text{add}(0, x) = x$$

$$\text{add}(n + 1, x) = \text{succ}(\text{add}(n, x))$$

Beispiel: Addition

Die Additionsfunktion $\text{add} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{add}(x, y) = x + y$ ist primitiv rekursiv.

$$\text{add}(0, x) = x$$

$$\text{add}(n + 1, x) = \text{succ}(\text{add}(n, x))$$

Genauer: Die Additionsfunktion $\text{add} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der Projektion $g(x) = \pi_{1,1}(x) = x$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{succ}(\pi_{3,2}(a, b, c))$:

$$\text{add}(0, x) = g(x)$$

$$\text{add}(n + 1, x) = h(n, \text{add}(n, x), x)$$

Beispiel: Multiplikation

Die Multiplikationsfunktion $\text{mult} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{mult}(x, y) = x \cdot y$ ist primitiv rekursiv.

$$\text{mult}(0, x) = 0$$

$$\text{mult}(n + 1, x) = \text{add}(\text{mult}(n, x), x)$$

Beispiel: Multiplikation

Die Multiplikationsfunktion $\text{mult} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{mult}(x, y) = x \cdot y$ ist primitiv rekursiv.

$$\text{mult}(0, x) = 0$$

$$\text{mult}(n + 1, x) = \text{add}(\text{mult}(n, x), x)$$

Genauer: Die Multiplikationsfunktion $\text{mult} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der konstanten Abbildung $g(x) = 0$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{add}(\pi_{3,2}(a, b, c), \pi_{3,3}(a, b, c))$:

$$\text{mult}(0, x) = g(x)$$

$$\text{mult}(n + 1, x) = h(n, \text{mult}(n, x), x)$$

Beispiel: Vorgänger

Die Vorgängerfunktion $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ mit $\text{pred}(x) = \max\{x - 1, 0\}$ ist primitiv rekursiv.

$$\text{pred}(0) = 0$$

$$\text{pred}(n + 1) = n$$

Beispiel: Vorgänger

Die Vorgängerfunktion $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ mit $\text{pred}(x) = \max\{x - 1, 0\}$ ist primitiv rekursiv.

$$\text{pred}(0) = 0$$

$$\text{pred}(n + 1) = n$$

Genauer: Die Vorgängerfunktion $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der konstanten Abbildung $g() = 0$ und aus der Projektion $h(a, b) = \pi_{2,1}(a, b) = a$:

$$\text{pred}(0) = g()$$

$$\text{pred}(n + 1) = h(n, \text{pred}(n))$$

Beispiel: Subtraktion

Die (modifizierte) Subtraktionsfunktion $\text{sub} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{sub}(x, y) = x \dot{-} y = \max\{x - y, 0\}$ ist primitiv rekursiv.

Da die zugrunde liegende primitive Rekursion über die erste Komponente läuft, definieren wir zunächst die Hilfsfunktion $\text{subAux}(y, x) = x \dot{-} y$.

$$\text{subAux}(0, x) = x$$

$$\text{subAux}(n + 1, x) = \text{pred}(\text{subAux}(n, x))$$

Genauer: Die Hilfsfunktion $\text{subAux} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht aus der Projektion $g(x) = \pi_{1,1}(x) = x$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{pred}(\pi_{3,2}(a, b, c))$:

$$\text{subAux}(0, x) = g(x)$$

$$\text{subAux}(n + 1, x) = h(n, \text{subAux}(n, x), x)$$

Danach definieren wir $\text{sub}(x, y) = \text{subAux}(\pi_{2,2}(x, y), \pi_{2,1}(x, y))$.